

Assignment 2 : Questions

Hill Climbing (2 Questions)

1. Hill Climbing on Course Selection

A student has 8 courses to choose from. Each course has a weekly workload and an expected score, as shown below:

Course	Workload (hrs/week)	Expected Score
1	6	70
2	14	65
3	10	88
4	8	75
5	12	80
6	4	50
7	15	78
8	5	60

The maximum workload allowed is 45 hours per week.

(a) Encode the problem for hill climbing. (b) Demonstrate two iterations of the Hill Climbing algorithm using any reasonable evaluation function.

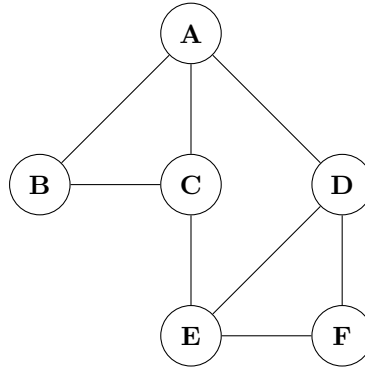


Figure 1: Graph for the Hill Climbing Graph Coloring Problem

The goal is to color the graph using the minimum number of colors such that adjacent vertices do not share colors.

(a) Encode the problem for hill climbing by defining states and neighborhoods.

(b) Perform two steps of the Hill Climbing algorithm starting from a random valid coloring.

Simulated Annealing (2 Questions)

1. Simulated Annealing for Task Scheduling

There are 9 tasks with processing times:

$$[3, 6, 2, 8, 5, 9, 4, 7, 1]$$

The tasks must be assigned to 3 identical machines. The objective is to minimize the makespan. Let the initial temperature be $T_0 = 120$, cooling schedule:

$$T(k) = T_0 \cdot 0.8^k$$

Demonstrate simulated annealing for 3 iterations starting from any reasonable initial assignment.

2. Simulated Annealing for Investment Allocation

A company must invest in 6 assets with returns:

$$[5\%, 3\%, 8\%, 6\%, 4\%, 7\%]$$

The total budget is \$60,000 and no asset may exceed 35% of total investment.

The objective is to maximize expected returns subject to constraints. Temperature update rule:

$$T(k) = \frac{100}{k + 1}$$

Perform 3 simulated annealing iterations, showing how candidate solutions are accepted or rejected based on probability.

Genetic Algorithm (2 Questions)

1. Genetic Algorithm for Warehouse Robot Routing

A warehouse contains 8 storage zones labeled Z_1 to Z_8 . A mobile robot must visit all zones exactly once to scan inventory. The travel-time matrix between zones is:

$$T = \begin{bmatrix} 0 & 12 & 25 & 18 & 30 & 22 & 27 & 35 \\ 12 & 0 & 20 & 28 & 26 & 30 & 18 & 24 \\ 25 & 20 & 0 & 16 & 14 & 32 & 29 & 33 \\ 18 & 28 & 16 & 0 & 21 & 24 & 31 & 40 \\ 30 & 26 & 14 & 21 & 0 & 35 & 22 & 28 \\ 22 & 30 & 32 & 24 & 35 & 0 & 26 & 30 \\ 27 & 18 & 29 & 31 & 22 & 26 & 0 & 19 \\ 35 & 24 & 33 & 40 & 28 & 30 & 19 & 0 \end{bmatrix}$$

Each chromosome is a permutation of the 8 zones representing the visiting sequence.

(a) Generate two initial parent chromosomes. (b) Perform one full GA iteration: selection, crossover (e.g., PMX or OX), and mutation.

2. Genetic Algorithm for Graph Bipartitioning

A graph consists of 10 vertices $\{V_1, \dots, V_{10}\}$ and edges:

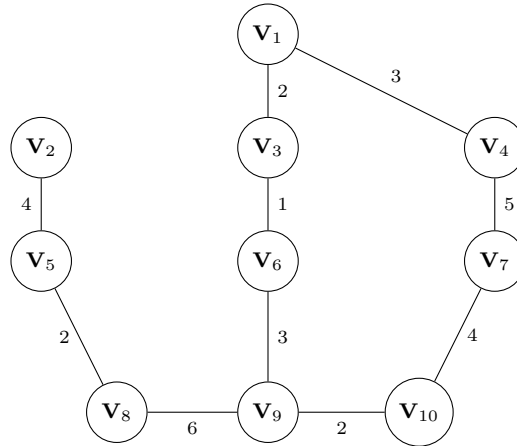


Figure 2: Graph with 10 vertices and specified edges

The objective is to divide vertices into two sets maximizing the number of edges between the sets. Chromosomes are binary strings encoding group membership.

Perform one GA iteration including: (a) Fitness evaluation, (b) Tournament selection, (c) Crossover and mutation.

Adversarial Search (2 Questions)

1. Tic-Tac-Toe: Minimax and Alpha-Beta Pruning

Consider the following mid-game Tic-Tac-Toe state where **X** is the maximizing player and it is **X's turn**:

X	O	X
O	X	
		O

Figure 3: Mid-game Tic-Tac-Toe state (X to move)

(a) Draw the complete game tree up to terminal states assuming optimal play from both players. (b) Apply the **Minimax algorithm** to compute

the utility of the root and determine the optimal move for X. (c) Redo the evaluation using **Alpha-Beta pruning**, clearly marking all cutoffs.

2. Alpha–Beta Pruning and Move Ordering

A two-player zero-sum game has the following game tree structure:

- The root is a MAX node with 2 children.
- Each child of the root is a MIN node with 2 children.
- Each of those nodes is a MAX node with 2 children.
- Each of those nodes is a MIN node with 2 children, whose children are terminal states.

Thus, the tree has depth 4 (root at depth 0, terminal nodes at depth 4), and there are $2 \times 2 \times 2 \times 2 = 16$ terminal states.

The tree is explored from left to right using the Alpha–Beta pruning algorithm.

Question A

The terminal node utilities (left to right) are:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]

Apply the Alpha–Beta pruning algorithm. Clearly show α and β updates, indicate all pruned branches, state how many terminal nodes are evaluated, and determine the final minimax value at the root.

Question B

Now consider the same tree structure, but the terminal node utilities (left to right) are:

[16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

Apply the Alpha–Beta pruning algorithm. Clearly show α and β updates, indicate all pruned branches, state how many terminal nodes are evaluated, and determine the final minimax value at the root.

Question C

- (a) Comparing the number of terminal nodes evaluated in Question A and Question B, are the final minimax values the same?
- (b) Explain why there is a difference in pruning efficiency even though:
 - The tree structure is identical.
 - The set of terminal utilities is identical.
- (c) What general conclusion can you draw about node evaluation order in Alpha–Beta pruning?